

SkVM: 重访跨异构 LLM 与代理框架的技能语言虚拟机

Le Chen Erhu Feng Yubin Xia Haibo Chen

上海交通大学, 中国上海

摘要

LLM 代理越来越多地将技能作为一种可复用的组合单元。尽管技能会在多种代理平台之间共享, 现有系统却把它们当作原始上下文, 致使同一技能在不同代理上表现不一致。这种脆弱性损害了技能的可移植性与执行效率。

为应对这一挑战, 我们分析了约 118,000 个技能, 并从传统编译器设计中获得启发。我们将技能视为代码, 将 LLM 视为异构处理器。为了使可移植性成为可操作的目标, 我们把技能的需求分解为一组原语能力, 并衡量每个模型-代理框架组合对这些能力的支持程度。基于这些能力画像, 我们提出 SkVM: 一个面向可移植、高效技能执行的编译与运行时系统。在编译时, SkVM 执行基于能力的编译、环境绑定和并发性提取。在运行时, SkVM 通过 JIT 代码固化与自适应重编译来优化性能。

我们在规模各异的八个 LLM 和三个代理框架上评估 SkVM, 覆盖 SkillsBench 与具有代表性的技能任务。结果表明, SkVM 在不同模型和环境显著提高了任务完成率, 同时最多减少 40% 的 token 消耗。在性能方面, SkVM 借助增强的并行性实现最高 3.2 $\times$ 加速, 并通过代码固化将延迟降低 19–50 $\times$ 。

1 引言

基于 LLM 的代理 [1, 37, 38, 11, 36, 54] 正在重塑软件任务的完成方式。开发者不再需要手工编写代码, 而只需描述目标, 代理便会推理、调用工具并交付结果 [58, 41]。为支持这一范式, 一个不断扩大的技能生态 [5, 6, 18, 50] 已经出现, 旨在让代理执行更加可靠、实用。技能主要由自然语言描述和脚本构成, 其中封装了特定领域的流程与最佳实践 [7]。目前已有超过 100,000 个技能分布于主要平台 [13, 50], 覆盖数据分析、金融、办公自动化、编程等众多领域。

然而, 当前代理对技能的支持十分简单: 系统把技能视为附加上下文并直接传给模型。不同模型在理解和执行技能方面存在显著差异 [33, 22, 30]。在横跨不同规模的八个模型中, 启用技能后, 总体有 15% 的任务性能下降 (例如 Opus 4.6 为 7%, Qwen3-30B 为 25%)。另有 17% 的任务分数保持不变 (不计完成率为 100% 的任务), 并且在多达 87% 的任务上, 至少有一个模型使用技能后没有取得提升。近期工作 [22] 得出了相近结论: 在 SWE-Benchmark 上, 49 个技能中有 39 个未带来分数提升, 另有 3 个造成明显的分数下降。

我们将这些问题归因于两个因素: (1) 即使技能已被加载, 模型在执行期间也可能忽略其指导; (2) 即使模型遵循技能, 技能对模型能力的假设也可能与模型的实际水平不符。除了任务完成增益上的差异, 当前技能中常见的半结构化代码片段和工作流式格式还会产生大量 token 开销 (例如 token 增加 451%, 而通过率保持不变 [22]) 并带来更高的执行延迟。

这一问题源于静态技能与底层模型及代理框架的多样性之间存在根本错配。技能所要求的能力可能与运行时调用的 LLM 所具备的能力不一致。回顾计算范式的演进 [51, 53, 15] (如图 1 所示), 我们观察到: 在代理时代,

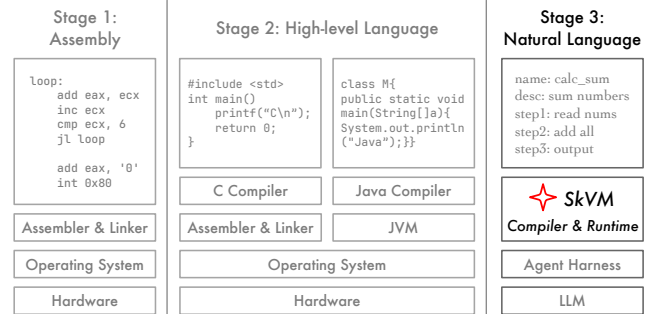


图 1: 编程抽象的演进。技能处于当前前沿: 它们是长达数百行的自然语言程序, 却缺少支持跨目标端可移植性的编译器与运行时。

技能就是代码, 而 LLM 就是处理器。但目前并不存在一种机制, 能够让技能在异构 LLM 与代理框架之间得到高效且可靠的执行。

受到经典计算系统处理代码方式的启发, 我们提出 SkVM, 一个面向技能的编译与运行时系统: 它支持高效的跨模型执行, 并为技能执行提供统一的底层支持。借助 SkVM, 技能会被编译为针对不同模型定制的形式, 使每个模型都能更好地理解 and 执行技能。此外, SkVM 提供统一的运行时环境, 对技能加载、解析与并发执行进行系统化管理。

我们围绕经典编译技术来组织 SkVM: 解释执行 [25]、提前 (AOT) 编译 [49, 48], 以及即时 (JIT) 优化 [3, 12, 52, 24, 15]。目前, 代理只使用解释执行来处理技能, 即把原始技能文本直接送入模型。这种方法妨碍了技能的可移植性与执行效率。相比之下, SkVM 进一步采用 AOT 和 JIT 编译, 针对不同的后端模型与执行环境进行优化:

AOT 编译: 技能内在地编码了对模型能力、环境配置和并行机会的需求。因此, SkVM 会在执行前分析这些需求并据此优化技能。基于能力的编译提取 26 项原语能力, 用它们刻画 LLM 与技能需求之间的能力缺口。每项能力以多个熟练度等级抽象模型行为的一个独立维度。编译器沿这些能力维度测量目标模型, 并调整技能规范, 使之更好地适配模型的优势与局限。其次, 环境绑定从技能描述中提取对软件包和工具的隐式依赖, 并生成可在加载时执行的设置脚本, 保证技能执行前所有依赖均已满足。最后, 并发性提取借鉴经典编译器对数据级并行 (DLP) [27]、指令级并行 (ILP) [45, 28] 和线程级并行 (TLP) [46] 的优化方法, 从技能中提取相应的三种粒度的并行机会, 并将其显式暴露给代理框架, 从而提高任务执行效率。

JIT 优化: 在运行时, SkVM 利用运行信息持续优化技能的性能与正确性。代码固化: 技能可能包含参数化脚本模板, 这些模板会在不同执行中被多次实例化。JIT 编译把这些高频模板具体化为已实例化的可执行代码, 从而绕过 LLM 解析。自适应重编译: JIT 监控会跟踪模型执行, 并在出现能力缺口时自适应地重编译技能, 使技

能够通过迭代优化实现自我改进。

技能编译完成后，技能运行时解析的是编译后的技能产物（包括优化后的技能、已实例化脚本和并发依赖图），而非简单地把原始文本传给模型。此外，技能运行时还会结合系统可用资源与工具能力，实时调度代理执行，保障执行过程稳定可靠。

我们在规模各异的八个 LLM 与三个代理框架上评估 SkVM，覆盖 SkillsBench [30] 和具有代表性的技能任务（共 118 个任务）。结果表明，SkVM 在不同模型与代理框架 [11, 38] 上显著提高任务完成率，平均提升 15.3%。此外，对于能够完成的任务，SkVM 最多可减少 40% 的 token 消耗。在性能方面，SkVM 通过细粒度并行化和 JIT 代码固化取得 $3.2\times-50\times$ 的端到端时间加速。

2 真实生态中的技能

本节首先介绍技能、使用技能的代理，以及技能在实践中的调用方式。我们从两个主要分发平台收集了超过 110,000 个技能，以大规模视角研究技能生态：clawhub.ai [13] (28,990 个技能) 与 skills.sh [50] (89,280 个技能)。在这一生态分析的基础上，我们进一步开展实验，识别当前技能使用中的具体问题。

2.1 技能定义

技能是一种可分发、自包含的知识包，用于增强 AI 代理处理某类特定任务的能力 [5, 2]。技能既可以引入基础模型原本不具备的知识，也可以在模型已有相关能力但本会即兴处理时，约束模型遵循规定的工作流。例如，一个 PDF 处理技能 [6] 会教模型如何使用 pdfplumber 提取表格，同时也约束模型在合并 PDF 文件时使用 pypdf，而不是已弃用的 PyPDF2。

一个技能通常定义在包含三层结构的文件中：（1）元数据—名称、描述和触发条件，使代理运行时能够自动发现并选择技能；（2）指令—技能主体，以自然语言编码多步工作流、工具引用及其他任务专用指导；以及（3）随附资源—存储在配套文件中的脚本、参考资料与模板。

这一结构使技能区别于临时的提示词片段。元数据使技能成为可发现、可选择的单元，令代理运行时能够自动把任务与适用技能匹配，而不必手工拼装提示词 [8]。与微调相比，技能无需修改权重，完全可以在提示词层面运行 [39, 32]，因而分发轻量且易于组合。多个代理平台已经趋同于这一抽象 [8, 16, 36, 2, 19, 14]，现有工具也已支持基础的技能编写、管理与优化。

2.2 LLM 代理与代理框架

LLM 代理在 ReAct 循环 [58] 中工作。它接收任务，推理下一步行动，发起工具调用（读取文件、运行命令、请求 API）[47, 35]，观察结果，然后再次推理。这一循环会不断重复，直至任务完成。

代理并不直接与外部世界交互。它运行在代理框架之内；该运行时框架位于模型与操作环境之间 [56, 29]。代理框架负责管理 LLM API 连接、注册代理可调用的工具、分派这些调用，并控制对话历史中有多少内容能够装入上下文窗口。Claude Code [11]、OpenCode [38] 和 OpenClaw [37] 都是广泛使用的代理框架。

代理框架之间存在会影响技能的差异 [20]。有些框架提供丰富的文件系统与网络搜索工具，另一些则只公开基本的读、写和 shell 执行操作。有些框架可以生成彼此独立、并发工作的子代理，另一些则只支持顺序执行。这些差异会直接影响技能指令能否得到执行。

2.3 技能特征分析

生态规模与分布。 技能生态规模庞大，但分布并不均衡。图 3 展示了两个平台上的下载次数。在 skills.sh 上，89,280 个技能中有 89% 的下载次数少于 86，只有少数头部技能达到 10^4-10^6 次。clawhub.ai 也呈现类似的长尾形态，峰值约为 10^2-10^3 次下载。两个平台合计存在超过 118,000 个技能，但其中绝大多数很少被使用。

技能分类。 技能为代理提供的内容差异很大。我们借助 LLM 分析，将下载次数超过 100 的 15,063 个技能按照技能最强调的方面分为三类：工具参考（52%）、流程指导（28%）和内容生成（20%）。

工具参考型技能教模型如何操作特定工具、API 或 CLI。它们本质上是加载到代理上下文中的使用文档。一个典型例子是说明如何使用 Python 合并、拆分和处理 PDF 文件的技能。

流程型技能规定执行任务时应遵循的逐步工作流与推理策略。一个典型例子是强制采用四阶段流程的调试技能：复现故障、追踪根因、施加针对性修复，并通过测试验证修复。

生成型技能要求模型生成内容、代码、文档或文章。其核心质量取决于模型自身的能力。例如，一个前端设计技能 [6] 会按照特定设计准则与样式惯例生成生产级 Web 组件。

工具参考型与流程型技能合计占整个生态的 80%。这些技能编码了可分析的结构，包括脚本片段和逐步工作流。执行这类技能是否正确，取决于代理能否执行规定步骤，而不是其生成能力。

工作流结构。 除了任务类型外，技能内部也存在重复结构。在 15,063 个已分类技能中，76% 包含显式的流程结构：编号步骤、条件分支，以及步骤间的数据依赖。

代码片段。 75% 的技能嵌入了类代码片段：shell 命令、API 调用模式，或整体形态在多次调用间重复、只有输入相关参数发生变化的脚本片段。例如，一个天气技能 [37] 可能为当前天气、天气预报与格式选项提供一组 curl 命令模板；命令结构保持不变，城市、输出格式和预报天数等参数则会变化。

2.4 当前技能使用中的问题

技能以原始文本形式加载到代理上下文中，不进行任何适配。但技能包含对模型、代理框架和用户环境的隐式假设。一旦这些假设不成立，便会出现三种故障模式。

P1：模型错配。 技能隐式假设模型的能力足以遵循其指令。实践中，不同模型的能力差异巨大 [33]。图 2 展示了三个模型和三个代理框架在一部分任务上的分数。¹ 在同一技能上，不同模型的性能差异显著。此外，启用技能后有 15% 的任务性能下降；17% 的任务分数保持不变（不计完成率为 100% 的任务）；并且在 87% 的任务上，至少有一个模型没有取得提升。

案例研究：pptx 创建技能 [6] 建议使用 PptxGenJS（一款 JavaScript 库）生成幻灯片。Claude-opus-4.6 和 gemini-3-flash 使用该技能时都得到 100 分，但 devstral-small 在 OpenCode 中会把 PptxGenJS 误读为 CLI，并反复执行错误命令。不使用该技能时，devstral-small 反而会使用 python-pptx，并得到 95 分。该技能假设模型能够区分库 API 与 CLI；这一假设对前沿模型成立，却不适用于较弱模型。

¹BareAgent 是我们构建的一个最小代理框架。它只实现基本的代理运行时逻辑，并且只公开读取、写入和执行工具。

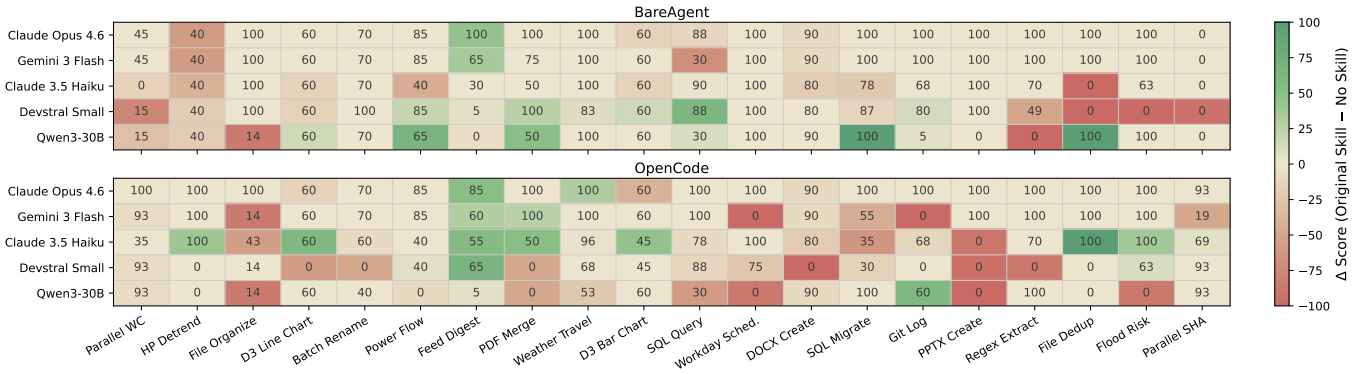


图 2: 不同模型与代理框架上的技能性能。列表示任务, 行表示模型, 每幅子图对应一个代理框架。单元格颜色表示相对于无技能基线的分数变化, 单元格标签给出任务的绝对分数。模型身份与代理框架选择都会显著影响结果。

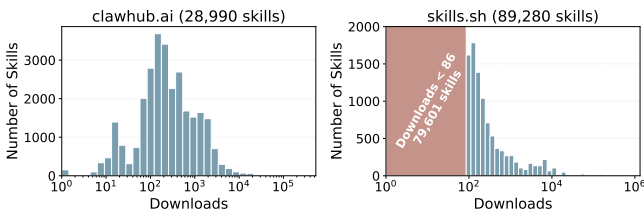


图 3: clawhub.ai 与 skills.sh 上的技能下载分布。两个平台都呈现长尾分布。

P2: 代理框架错配。 同一模型在同一任务上的结果会随其运行的代理框架而变化 [57]。图 2 也展示了这一维度: 在整个热力图中, 代理框架造成的方差与模型造成的方差相当。技能在编写时并不知道代理框架会提供哪些工具和插件。

案例研究: Gemini 3 Flash 使用原始技能在 BareAgent 上执行工作日排程任务时得到 100 分, 但在 OpenCode 上使用同一技能却得到 0 分。故障由一个格式错误的 JSON 键触发, 它使输出无法解析。BareAgent 只添加一条最小系统提示词, 而 OpenCode 会在前面加入大量工具文档; 更长的组合上下文诱发了这一格式错误。

P3: 环境错配。 技能依赖于执行环境, 但用户机器上可能缺少所需的软件包、工具或配置。一些技能会列出前置条件, 但这些指令往往较为笼统, 面向通用机器, 而非用户的实际设置。真正的需求取决于当前操作系统、硬件、已安装版本及软件包管理器状态。还有一些技能会完全遗漏依赖。图 4 展示了两个代表性任务受到的影响。缺少所需库时, 两个 Qwen 模型的成功率都降至 33–67%, 并在失败的变通尝试中生成 2–4× 更多输出 token。即便 Claude Opus 4.6 每次都能成功, 其输出 token 仍增加 56–69%, 因为它必须在运行时诊断错配并安装缺失的软件包。因此, 把诊断留给模型, 会让每项缺失依赖都反复损害正确性与效率。

2.5 挑战

技能以纯 Markdown 形式分发, 使任何代理都能在无专用工具的情况下加载它们。上述错配破坏了这一可移植性承诺。恢复可移植性需要针对每个目标端适配技能, 而这面临两个挑战。

C1: 目标端在多个维度上不同。 上述模型、代理框架与环境三个维度彼此独立变化。一个需要沿某个维度进行适配的技能, 在两个维度共同变化时可能需要另一种适

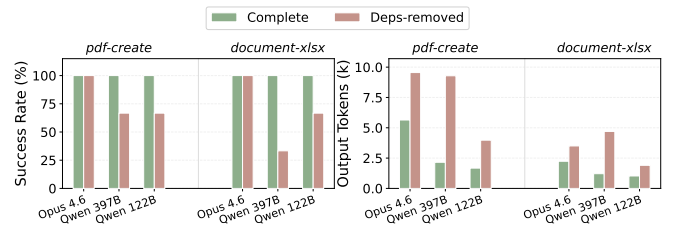


图 4: 移除一项必要依赖会同时损害正确性与效率。

配。对于运行在功能丰富框架上的弱模型, 正确修复方式不同于同一模型运行在最小框架上的修复方式。这种交互使适配空间呈组合式增长: 一套固定重写规则无法覆盖它。

C2: 技能是非结构化自然语言。 源代码在目标系统上运行前需要编译, 并且具有可供工具分析的语法、类型和明确定义的 AST [3]。相比之下, 技能以自然语言编写。尽管技能大多遵循共同惯例创建, 不同技能在结构、术语和细节层次上仍有很大差异。在进行任何适配前, 都必须先从文本中恢复技能的隐式需求与工作流结构。

3 设计概览

SkVM 是一个面向技能的编译与运行时系统。它将安装时对技能进行特化的 AOT 编译器 [3] 与解决执行时不确定性的运行时结合起来, 使技能能够跨异构目标端移植。图 5 展示了其架构。

AOT 编译器。 安装技能时, 编译器针对目标环境分析该技能, 并生成一个或多个优化后的技能变体。编译器执行三个彼此独立的阶段: 基于能力的编译 (§4.1) 通过使技能适配目标端可用的能力, 处理模型与代理框架的错配 (P1–P2)。环境绑定 (§4.2) 通过具体化执行所需的依赖, 处理环境错配 (P3)。并发性提取 (§4.3) 识别技能工作流中潜藏的并行性, 并将其映射到目标代理框架。编译后的变体会按目标端分别存储。

运行时与 JIT 优化器。 任务到来时, 运行时会选择为当前 (模型, 代理框架) 组合编译的变体, 并通过标准的渐进式披露机制加载它。随后, 两个组件在常规执行之上发挥作用。JIT 优化器 (§5.2) 跨调用监控执行结果, 并在检测到 AOT 编译器未发现的能力缺口时触发重编译。它还会把结构固定的代码模式编译成可执行函数, 完全绕过 LLM 推理。资源感知调度器 (§5.3) 连接编译时并行性标注与运行时资源可用性; 当需求超出容量时, 它会节流或挂起并发子代理。

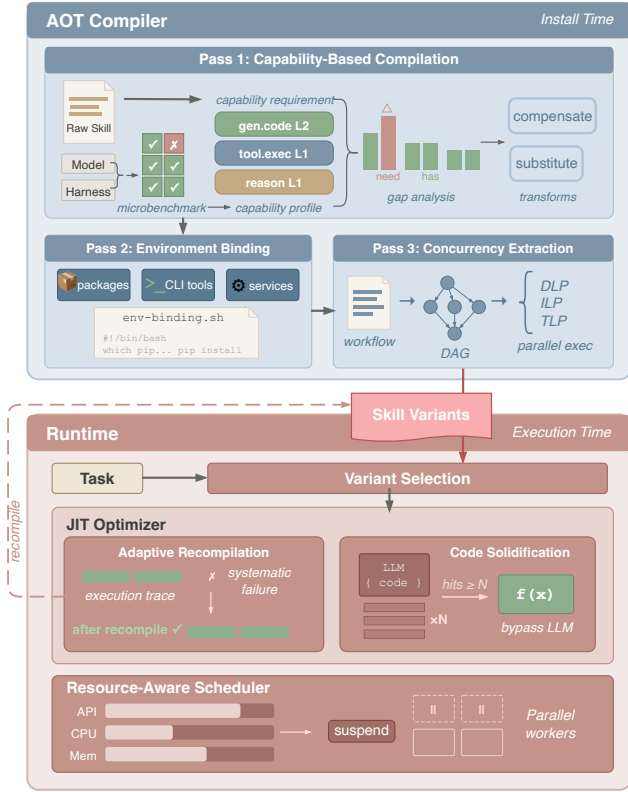


图 5: SkVM 架构。AOT 编译器在安装时经过三个编译阶段生成优化的技能变体；运行时则在执行期间负责变体选择、JIT 优化和资源感知调度。

4 技能编译

当用户首次安装技能时，SkVM 会在技能尚未运行前，针对目标端（模型、代理框架、主机环境）编译该技能。编译器读取原始技能文本，识别目标端，并经过三个顺序阶段生成一个或多个优化后的技能变体。每个阶段弥补一种不同的缺口，即技能预期与目标端实际提供的内容之间的差异。

4.1 基于能力的编译

第一个阶段处理模型与代理框架的错配。由于目标端差异很大 (§2.5)，编译器不能内置针对每个目标端的优化逻辑。为此，SkVM 引入原语能力，作为一种抽象词汇，用于表达技能需要什么，以及目标端能够提供什么。编译器离线使用针对这些能力的微基准对每个目标端进行画像，将画像结果与技能的能力需求比较，并根据两者之间的缺口选择优化策略。

4.1.1 原语能力

正如 JVM 字节码定义 Java 程序要求其运行时提供的基本操作，原语能力定义技能要求其目标端提供的基本能力。原语能力是一个不可再分的单元，用来描述完成技能所定义任务所需的能力。它完全从需求侧定义，与任何特定模型无关。原语能力的定义遵循三个原则：

- **可组合性：**技能中的任何具体任务都能表示为若干原语能力的组合。
- **通用性：**每项原语能力都必须足够宽泛，能够出现在许多技能中，从而保持能力总量较小并限制编译器的工作量。
- **语义独立性：**原语能力关注结构正确性，而不判断输出是否深刻或论证是否充分。

表 1: 代表性原语能力及其熟练度等级。完整目录包含分属四类的 26 项原语能力。

原语	L1	L2	L3
gen.code.shell	基本命令 (ls, cat)	管道、重定向、循环	复杂管道 (sed, awk)
reason.arithmetic	单步运算	多步运算	复合运算
tool.exec	单条命令	参数与相对路径	链式多步执行
follow.procedure	3 个顺序步骤	5-7 步且包含分支	循环与验证

我们分两个阶段从 15,063 个技能构成的语料库中推导出原语能力集合。首先，我们选取涵盖文档生成、脚本执行、数据分析和代码开发的 50 个代表性技能，并在上述三项原则指导下借助 LLM 分析，将它们分解为初始的 19 项原语能力。随后，我们人工审查每项原语能力，确认其符合全部三项原则。其次，我们用完整的 15,063 个技能语料库验证初始集合，检查每个技能的需求能否表示为已有原语能力的组合。无法覆盖的技能会暴露出缺失能力。只有当一项缺失能力出现在至少 1% 的语料中时，我们才把它加入集合，从而过滤罕见或特定领域的需求。这一过程最终收敛到分属四类的 26 项原语能力，可覆盖 95% 技能的需求。

我们还观察到，不同任务对同一原语能力的深度要求不同。因此，我们为每项原语能力定义多个熟练度等级。更高等级代表对该能力更深入的使用： $L(n)$ 能力无法满足 $L(n+1)$ 需求。表 1 给出了代表性示例。对于原语能力的每个等级，我们生成一组微基准，并在不同模型上进行评估。如果某模型通过了更高等级的微基准，却未通过较低等级的微基准，我们会把这种不一致解释为等级层次校准不当的证据，随后修订等级定义并重新评估。

4.1.2 测量技能与目标端之间的缺口

应用优化之前，编译器必须确定技能需求与目标端所提供能力之间的缺口。在技能一侧，编译器会在安装时提取技能的能力需求：它把技能分解为具体任务，并将每个任务映射到所需的原语能力。

在目标端一侧，SkVM 使用为每项原语能力生成的微基准对目标端进行画像，以衡量目标端对该能力的支持程度。如果目标端通过了某项原语能力给定等级的微基准，就表示模型在该能力上至少达到了这一熟练度等级。这种画像在用户设置代理和模型时执行。结果会被缓存，并复用于同一目标端的所有技能编译。

4.1.3 能力感知的技能变换

获得技能的能力需求与目标端的能力画像后，编译器会遍历每项能力需求，并根据所需等级与目标端画像等级之间的缺口选择变换。

当目标端支持所需能力但等级较低时，采用补偿。编译器首先识别同一原语能力两个等级之间的差异，再据此变换技能，将其能力需求从较高等级降低到目标模型支持的等级。这类变换可以包括提供示例、使指令更加明确，以及强化任务约束。补偿是首选变换，因为它保留了技能的原始意图。

当补偿不可行时，替代是后备方案：目标端完全缺少该能力，或等级缺口过大，无法通过补偿可靠弥合。编译器会切换到另一条实现路径，用不同能力达成同一目标。例如，如果某技能的 pandas 工作流程要求 L3 的 gen.code.python，而目标端只达到 L1，那么当目标端的 gen.code.sql 达到 L2 时，编译器可以切换到基于 SQL 的路径。这些替代路径形成等价类，由编译器在需求提取期间识别。

案例研究。 图 6 展示了一个真实技能的编译过程；该技能教代理使用 python-pptx 创建 PPTX 演示文稿。这个技能要求五项不同等级的原语能力。目标端与其中四项

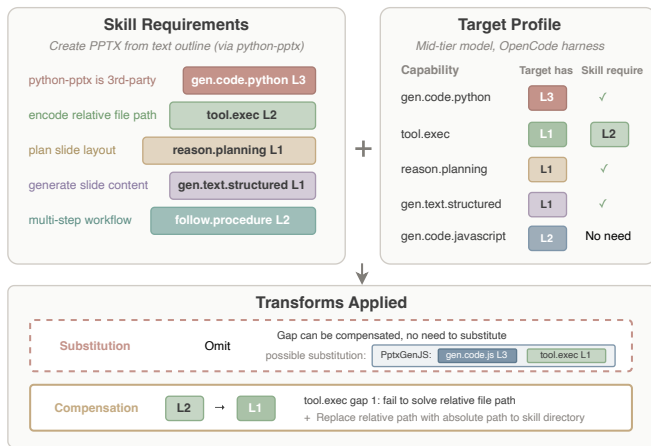


图 6: 对 PPTX 技能进行基于能力的编译。编译器提取技能需求 (左), 对目标端进行画像 (中), 并根据缺口应用变换 (下)。

直接匹配, 但 tool.exec 只达到 L1, 而技能需要 L2 才能解析相对文件路径。

编译器识别出一条可能的替代路径: 经由 *gen.code.javascript* 切换到 *PptxGenJS*, 即可替换 *python-pptx*, 并把 *tool.exec* 的要求降低到 L1。不过, 由于 *tool.exec* 的缺口只有一个等级, 补偿仍是首选。L2 画像结果显示, 该模型在执行命令时无法解析相对文件路径。于是编译器把绝对路径解析注入编译后的技能中, 用指向技能目录的绝对路径替换相对路径。

4.2 环境绑定

第二个阶段解决 P3: 环境错配。编译器不只检查依赖是否已经安装, 而是使用内置的环境绑定技能, 使技能假设与主机环境相协调, 并生成环境绑定脚本。

安装技能时, 编译器首先从第一阶段输出的技能与所有显式前置条件中提取依赖清单, 其中包括外部库、CLI 工具和系统服务。对于清单中的每一项, 编译器都会运行轻量级存在性检查 (例如 *pip show*、*which*)。已经满足的依赖无需进一步处理。检查失败时, 编译器会执行更深入、感知系统的探测, 以确定如何解决该依赖。随后, 编译器生成幂等的环境绑定脚本; 该脚本会在技能执行前检查并修复依赖。当环境发生变化或环境绑定脚本失败时, 执行会回退到模型, 并把脚本输出作为附加上下文传给模型。

4.3 并发性提取

第三个阶段寻找隐藏在显式顺序描述中的并行性。回顾前文, 76% 的技能包含流程结构 (§2.3)。这些技能用顺序文本描述多步工作流, 但并非每一步都真正依赖前一步。

编译器借助 LLM 分析把技能分解为离散步骤, 并识别每一步消费哪些输入、产生哪些输出。随后, 它构建工作流 DAG: 节点表示步骤; 若步骤 B 消费步骤 A 的输出, 则存在一条从 A 指向 B 的边。编译器还会分析每个步骤内部可并发执行的子操作。图 7 用一个事件响应技能展示这一过程。对于步骤之间或步骤内部的每个并行机会, 编译器都会根据任务结构与代理框架选择并发原语。SkVM 定义了三个并行层级。

数据级并行 (DLP)。单个步骤对多个独立数据项应用同一操作, 例如对 15 个 CSV 文件分别运行相同分析。编译器重写该步骤, 使用适合技能实现方式的语言级并行原语来并发处理各数据项, 例如 shell 级并行执行、Python

的 multiprocessing, 或 JavaScript 的 Promise.all。DLP 不需要代理框架提供特殊支持。

指令级并行 (ILP)。多个彼此独立的步骤各自需要工具调用, 并且它们之间没有数据依赖, 例如对一个项目运行八个独立的代码分析脚本。编译器把相应的 DAG 节点归入同一个并行阶段, 并重写技能, 使这些工具调用在一个 LLM 轮次中一并发出, 而不是按顺序文本逐个发出。编译器还会在技能中标注: 批处理完成后, 应将哪些步骤输出绑定回哪些下游 DAG 输入。ILP 要求代理框架支持批量工具分派。

线程级并行 (TLP)。工作流可以分解为需要多轮推理的独立子任务, 例如调试三个彼此独立的服务。编译器重写技能, 把每个子任务提取为单独的子代理块; 每个块都带有明确的任务描述、声明的输入上下文, 以及可由父工作流消费的预期输出。标注会将该块标记为交由子代理分派, 运行时因此在各自会话中为每个块生成一个子代理, 并把返回的输出合并回 DAG。TLP 要求代理框架支持生成子代理。

若目标代理框架不具备某个机会所需的原语, 该机会就回退为顺序执行。对于每个可行机会, 编译器要么重写技能指令, 让数据项并发执行 (DLP), 要么生成执行标注, 使并行结构显式化: ILP 使用跨 DAG 节点的并行阶段, TLP 使用带输入、输出契约的子代理任务块。运行时读取这些标注, 并通过对应的代理框架原语进行分派。

5 采用 JIT 优化的技能运行时

AOT 编译会为不同目标端生成多个技能变体。因此在运行时, SkVM 会加载为当前目标端编译的技能变体。由于每个技能与目标端 (模型 + 代理框架) 组合只执行一次 AOT 编译, 编译开销较小。

不过, 有些问题只有到执行时才会浮现, 例如静态画像遗漏的能力缺口, 或并行子代理争用受 API 速率限制约束的资源。另一些机会, 例如在多次调用中反复出现的代码模式, 只有重复运行后才会显现。SkVM 运行时通过 JIT 优化 (§5.2) 和资源感知调度器 (§5.3) 处理这两类情况。

5.1 技能注册与执行

编译会为每个技能生成多个技能变体, 以及阶段 2 生成的环境绑定脚本。新任务到来时, 运行时会选择为当前 (模型, 代理框架) 组合编译的变体。从代理的角度看, 多个变体带来的复杂性是不可见的: 它看到的仍是一组技能, 并通过 §2 所述的同一种渐进式披露机制加载它们。在加载完整技能上下文之前, 运行时先执行环境绑定脚本, 验证运行时依赖是否已满足。

5.2 JIT 优化

AOT 编译会针对每个目标端调整原语能力; 在此基础上, SkVM 进一步应用 JIT 优化, 通过自适应重编译与代码固化这两种互补机制, 提高执行质量与效率。

5.2.1 自适应重编译

运行时跟踪使用技能执行的每个任务的结果。当技能执行失败 (例如代理中止或收到人工反馈), 或在代理循环中发生重试时, 运行时记录结构化的失败或重试日志。尽管当前模型可能通过反思进行自我纠正, 但纠正过程不会被记录, 因此后续执行仍可能遇到相同错误。

当同一技能在多次调用中失败时, 运行时分析这些失败究竟是任务特有的, 还是反映技能本身存在系统性能力缺口。只有后一种情况才会触发自适应重编译: 运行时把累积的失败日志和模型的自我恢复轨迹作为编译器

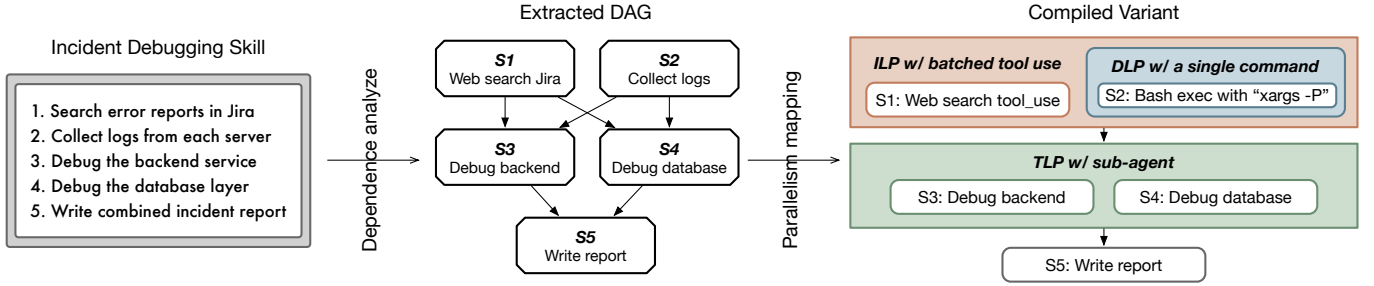


图 7: 在事件响应技能上进行并发性提取。编译器将顺序步骤分解为工作流 DAG，识别步骤之间和步骤内部的并行机会，并把每个机会映射到相应的并发原语。

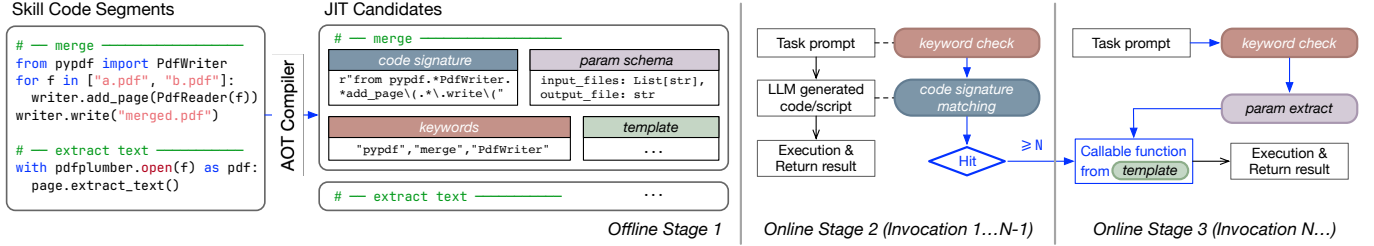


图 8: 代码固化流水线。AOT 编译器分析技能中的代码段，并生成带代码签名与模板的 JIT 候选项（阶段 1）。运行时跨多次调用验证预测（阶段 2）。候选项晋升后，编译后的函数会替代 LLM 推理（阶段 3）。

输入，由编译器对技能施加针对性的补偿变换。如果重编译后的变体表现更差，运行时就回滚到前一版本。后续每次重编译都从迄今观察到的最佳变体开始，以确保优化能够改善技能性能。

5.2.2 代码固化

回顾前文，75% 的技能包含可固化代码 (§2.3)：这类代码整体结构固定，只有输入相关参数发生变化。在常规执行中，LLM 每次调用都会重新执行完整的推理和工具使用循环，把 token 与延迟花在结构完全相同的过程上。代码固化会识别这些重复模式，并将其晋升为绕过模型推理的可执行代码。图 8 展示了包含三个阶段的流水线。

在第一阶段，AOT 编译器分析每个技能中的代码和脚本片段，判断它们是否包含适合固化的参数化模式。对于符合条件的片段，编译器会生成一个包含四部分的 JIT 候选项：用于筛选相关调用的关键词；以匹配模式刻画预期输出结构的代码签名；带参数槽位的代码模板；以及描述可提取参数的参数模式。这些候选项会与编译后的技能变体一同打包。

在第二阶段，运行时监控器观察多次重复调用中的 LLM 调用。对于每次调用，它先检查关键词以判断相关性，再将生成的代码与代码签名匹配。只有当代码签名在连续多次调用中成功匹配后，系统才会触发代码固化，从而保证 LLM 生成的代码结构与 AOT 阶段分析出的模式保持一致。如果模型输出持续偏离代码签名，系统会继续采用安全的 LLM 路径，绝不晋升该候选项。

在第三阶段，模板被实例化为可调用函数：系统把模板代码与参数提取、输出处理封装在一起，生成独立的 shell 脚本或代码函数。由于 AOT 阶段已经完成分析，这一步实例化轻量且机械化。后续调用会完全绕过 LLM：运行时从任务上下文提取参数，调用实例化函数并返回结果。如果固化代码的执行导致代理任务失败或遇到运行时异常，SkVM 会触发后备机制，重新启用基于 LLM 的代码生成以保证正确性。

5.3 资源感知的并行调度

在 AOT 编译的阶段 3 中，编译器会识别可能采用并行的位置；但多大的并行度才高效，取决于运行时不断变化的条件：CPU 使用率、外部 API 速率限制、机器可用内存，以及其他共享资源。

SkVM 动态监控系统状态并据此调度任务执行，弥合编译时机与运行时现实之间的差距。对于受 API 约束的工作负载，它监控响应延迟与 HTTP 429（速率限制）信号；对于计算密集型工作负载，它监控 CPU 与内存使用率。当压力超过阈值时，SkVM 会采用两种机制缓解资源争用。首先，它对新子代理的启动实施节流，避免引入更多并发压力。其次，对于已经运行的子代理，SkVM 会根据每个代理的资源使用量或启动顺序，有选择地挂起其中一部分（即暂停其代理循环执行），以减少对共享资源的活跃争用。SkVM 还会记录每个技能上次在当前系统运行时的有效并发度，并将其作为后续执行的并发提示。

6 评估

6.1 实验设置

基准。我们选择并扩展现有技能基准，包括 Skills-Bench [30] 和 PinchBench [40]，覆盖代码生成、数据分析、文档创建和系统管理等多种代理任务。每个任务至少配有一个来自主流技能来源的技能：Anthropic-skills [6]、OpenClaw skills [37] 和 skills.sh [50]。我们采用自动化评估套件，根据预定义标准进行静态分析与基于 LLM 的评估 [26, 59]。

模型。我们在分属三个能力层级的八个模型上评估 SkVM，以全面考察它在不同模型能力下的有效性。SOTA: claude-opus-4.6 [10]、deepseek-v3.2 [31]。中型: gemini-3-flash [17]、qwen3.5-397b [44]、qwen3.5-122b [43]、claude-3.5-haiku [4]。小型: qwen3-30b [42]、devstral-small [34]。

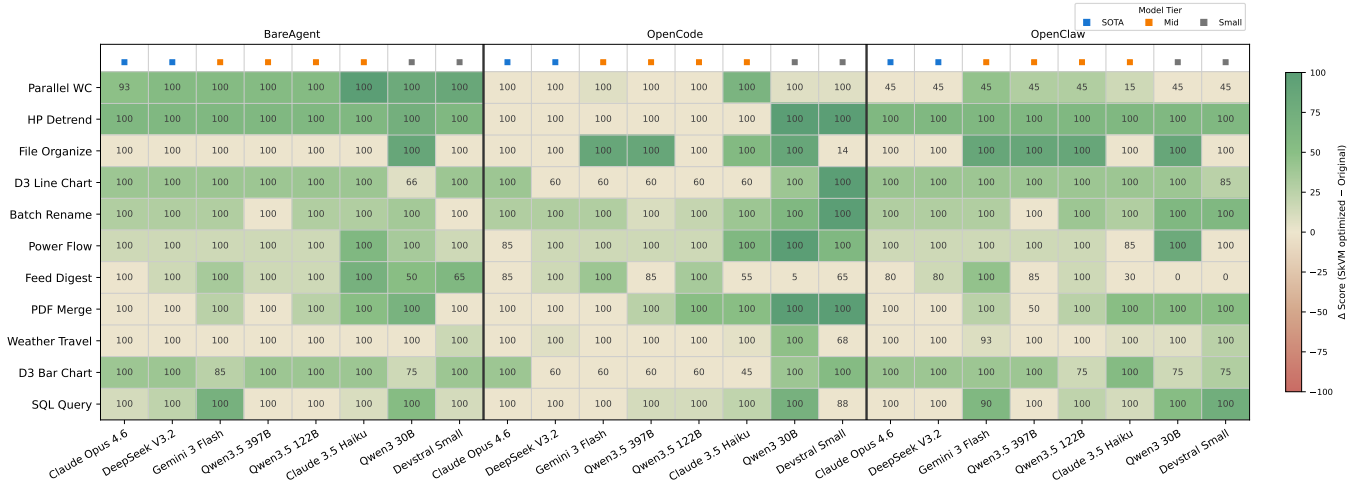


图 9: 技能编译的有效性。单元格数值表示经 SkVM 优化后的任务完成率；单元格颜色表示相对于原始基线技能的分數提升（绿色）或下降（红色）。

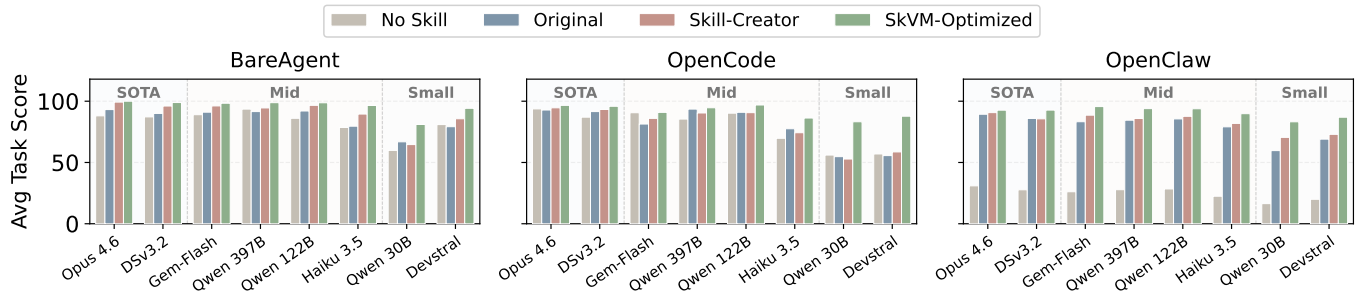


图 10: 各技能变体的平均任务分数。在八个模型与三个代理框架上比较四种变体：无技能、原始技能、Skill-Creator 和 SkVM 优化技能。图中用底色区分 SOTA、中型与小型模型。SkVM 优化技能始终取得最高分数，并且在较弱模型与 OpenClaw 上获得最大的绝对增益。

基线。 我们将 SkVM 与三个基线比较。无技能：代理在未加载任何技能的情况下执行。原始技能：使用原始技能。*Skill-Creator*：使用 Anthropic 的 Skill-Creator [9] 配合 claude-opus-4.6 优化后的技能。

代理框架。 我们选择三个不同的开源代理框架，以评估 SkVM 在多样化执行环境中的性能。各框架提供不同功能：*BareAgent* 是一个最小框架，不引入额外抽象层，直接把编译后的技能注入系统提示词。*OpenCode* [38] 是一个功能完整的代码代理框架，支持批量工具分派，并提供丰富的代码执行工具集。*OpenClaw* [37] 是一个通用代理框架，集成了支持复杂多步任务执行的功能模块。

方法。 我们首先对不同模型的原语能力进行画像，然后应用 SkVM 的 AOT 与 JIT 编译，为每个模型生成定制化的优化技能变体。为了支持编译后的技能产物，我们用自定义加载与管理机制扩展代理框架，使其能够执行 SkVM 的优化。对于每项任务，我们生成五个多样化输入实例，并测量平均任务完成率、token 消耗与端到端执行延迟。所有实验均在配备 16GB 内存的 Mac Mini M4 上进行。

6.2 技能编译的有效性

我们首先评估 SkVM 提高任务完成率的有效性。我们在不同任务、模型和代理框架上，将 SkVM 优化的技能与未优化的基线技能比较。对于 SkVM，主要增益来自于能力的编译与 JIT 优化。图 9 展示了八个模型和三个代理框架上的编译效果。单元格数值表示 SkVM 优化后

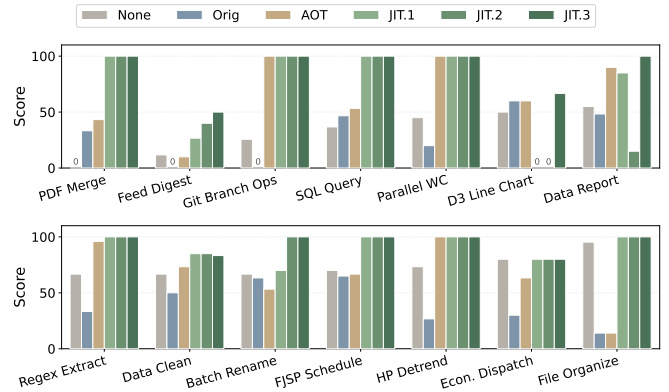


图 11: 14 类技能上的分阶段优化拆解。每组展示六个阶段的分数：无技能、原始技能、AOT 编译技能，以及经过三轮 JIT 优化的技能。

的任务完成率；单元格颜色表示相对于未优化基线技能的分數提升（绿色）或下降（红色）。

图 9 展示了 SkVM 相对于原始技能，在多种模型与代理框架上的优化效果。较弱模型从 SkVM 优化中获益更多。这一发现表明，对于大多数代理任务，较弱模型已经具备处理逻辑组成部分的足够能力，但不擅长生成复杂 JSON 结构或管理环境依赖等非逻辑方面。SkVM 的编译优化通过细化原语能力来处理这些缺口，从而提高总体任务完成率。对于较强模型，由于它们已经能够胜任

任务，提升相对有限；SkVM 优化主要减少 token 消耗与执行延迟。图 10 针对每个模型与代理框架，汇总了全部四种技能变体的平均任务分数。在每一种模型-代理框架组合上，SkVM 优化技能都取得最高分数。随着模型能力降低，相对于 Skill-Creator 基线的提升会扩大：在 BareAgent 上，SkVM 优化技能相对于 Skill-Creator，在 qwen3-30b 上提升 25%，在 devstral-small 上提升 10%。就性能下降而言，SkVM 编译技能只在 4.5% 的任务上降低任务完成率，而原始技能的对应比例为 15%。使用原始技能时，OpenCode 与 OpenClaw 之间的差异会随模型变化，最高达到 13 个百分点，这表明仅代理框架行为本身就会实质性改变结果。SkVM 优化会同时提高两个框架上的分数，并将这种跨框架差距缩小至最多 5 个百分点。

6.3 分阶段优化拆解

图 11 分析了 Qwen3-30B 配合 BareAgent 时，各优化阶段对 14 类技能的贡献。每组展示六个阶段的分数：无技能、原始技能、AOT 编译技能，以及最多三轮 JIT 优化。各轮所用任务类型相同，但内容不同。

在 14 类技能中，有 11 类使用原始技能时的表现不如无技能。经过 SkVM 的 AOT 编译，平均任务分数提高 88%。JIT 优化进一步识别 AOT 编译期间未暴露的能力缺陷，继续改善性能。经过一轮 JIT 后，14 个技能中有 8 个在评估任务上取得满分；三轮后，有 10 个取得满分。

JIT 优化也可能造成性能下降。例如，在折线图任务中，JIT 编译器向技能添加了示例文件，但该文件过长并触发解析错误，致使分数下降。经过由先前失败日志引导的三轮修订，JIT 编译器最终修复了该问题。

6.4 编译带来的 Token 与成本效率

除了提高任务成功率，SkVM 还通过减少 token 消耗，为 SOTA LLM 带来显著的成本收益。LLM 可以在迭代式代理循环中根据环境反馈调整行动，从而纠正执行错误。因此，即便任务最终成功，这一方法也可能浪费大量 token。图 12 展示了 SkVM 编译后，不同模型层级与代理框架上的任务成功率和 token 成本如何变化。

评估显示，对于大多数模型与代理框架组合，SkVM 编译会同时提高任务成功率并减少 token 消耗。对于最强模型与最弱代理框架的组合（DS-v3.2 + BareAgent），我们观察到接近 40% 的 token 节省。产生这一提升的原因是：代理循环在纠正环境相关与工具调用错误时会产生交互开销，并且常常需要多次重试。借助 JIT 编译，SkVM 让模型完全避开这些错误路径，消除冗余交互并节省大量 token。

对于较弱模型，SkVM 偶尔可能消耗更多 token。这是因为 SkVM 提高了任务完成率，使代理执行轨迹比使用基线技能时更长，因而在边缘情形下会小幅增加 token 用量。

6.5 目标端画像的开销

基于能力的编译要求对目标端进行一次画像，以构建其能力画像 (§4.1)。这种画像对于每个（模型，代理框架）组合只产生一次成本，结果会被缓存，并复用于同一目标端后续的所有技能编译。我们在两个小型模型 devstral-small 和 qwen3-30b 上测量了这一开销。

图 13 展示了画像时长与成本按类别拆解的结果。覆盖全部原语能力的完整画像，在 devstral-small 上用时 7.3 分钟，在 qwen3-30b 上用时 31.1 分钟，成本分别为 \$0.033 和 \$0.079。

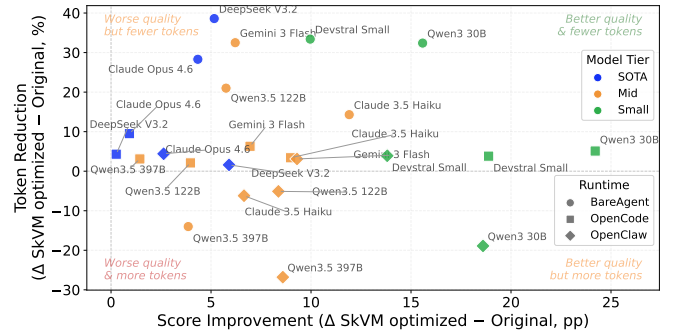


图 12: 相对于原始技能基线，SkVM 优化带来的分数增益与 token 节省。每个点表示一个（模型，代理框架）组合；颜色表示模型层级，形状表示代理框架。大多数点位于右上象限：质量更高，token 也更少。

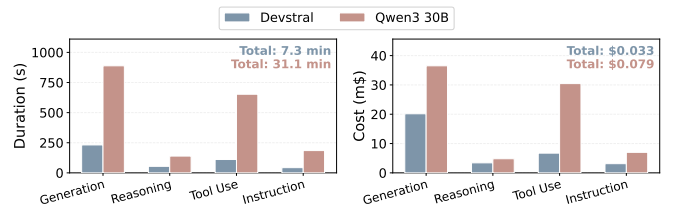


图 13: 两个小型模型的能力画像开销，按类别拆解。左：时长（秒）；右：成本（毫美元）。

6.6 环境绑定评估

前两个实验都在依赖完备的环境中进行测试。然而在真实技能场景中，代理执行时缺少环境依赖十分常见。解决这类环境问题对于较弱 LLM 构成显著挑战，并且经常导致技能执行失败。SkVM 在 AOT 编译阶段通过环境绑定机制处理这一问题。该机制把环境依赖安装从 LLM 执行阶段前移到执行前的预处理阶段，使 LLM 只需专注于技能逻辑。

图 14 展示了不同模型在三种环境配置下的性能：完整环境、采用环境绑定，以及缺少依赖。claude-opus-4.6 等较强模型可以自行从依赖缺失中恢复，但代价是消耗更多 token。对于 qwen3.5-122b 等较弱模型，依赖缺失会导致执行失败。环境绑定能够完全恢复执行正确性，并大幅减少 token 消耗。

6.7 并发性提取评估

SkVM 从技能内部提取更多并行机会并提高效率。我们评估三种并行类型：数据级并行（DLP）、指令级并行（ILP）和线程级并行（TLP）。当单个技能内的指令批量处理大量独立数据时，可以应用 DLP。当技能内有多条独立指令或代码段可并发执行时，可以应用 ILP。当一个技能内运行多个独立子代理，且每个子代理处理无交叉依赖的自包含子任务时，可以应用 TLP。图 16 展示了 SkVM 在这三种并行类型上的性能提升。

实验结果表明，SkVM 的并行性提取策略实现最高 3.2× 的端到端加速。TLP 的平均提升最大，因为其更粗粒度的并行性带来更显著的优化效果。ILP 在指令层工作，能够同时减少执行时间与 LLM 调用次数。DLP 的收益完全来自数据并行，因此性能提升会随数据并行度直接扩展。我们还观察到，由于 LLM 吞吐波动以及代理循环迭代次数不同，代理执行本身存在变化。因此，合理范围内的计时波动是可以接受的。

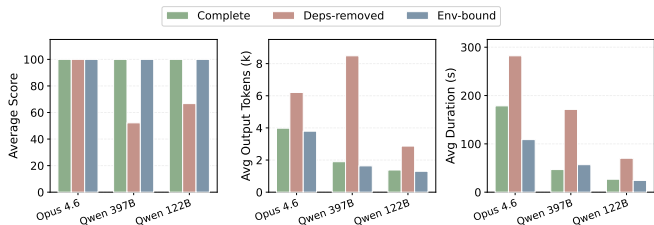


图 14: 环境绑定恢复了正确性、token 效率与执行速度。从左到右依次为: 各模型在两个任务上的平均分数、平均 token 用量与平均时长。对三个模型而言, 环境绑定后的性能都恢复到完整环境的水平。

6.8 代码固化评估

我们进一步分析: 采用 JIT 优化中的代码固化机制, 能否准确识别可复用代码段并提高代理执行效率。如图 15 所示, 我们对两个典型技能 (天气、文档 PDF) 进行详细分析。对于两个 PDF 任务, 尽管任务输入不同, LLM 生成的代码都与代码签名匹配。因此, SkVM 可以应用 JIT 编译优化, 直接使用模板与输入参数生成固化代码。对于 PDF-extract 任务, JIT 优化把执行时间从 10,469–15,116 ms 降至 206–568 ms, 实现 19–50 $\times$ 加速。在两个天气任务中, weather-current 需要调用外部天气 API, 其加速受网络延迟限制: 执行时间从 9,000 ms 降至 2,000 ms, 提升 5 $\times$ 。相比之下, weather-forecast 使用更灵活的格式, 因此生成的代码结构偏离代码签名, 无法匹配。于是, 全部八次调用都依赖 LLM 生成, 以避免调用错误代码。

如果固化代码执行导致任务失败, SkVM 的安全机制会触发回退, 重新启用 LLM 代码生成以保证正确性。

7 讨论

技能编译中的非确定性。 与传统程序编译不同, 技能编译以自然语言为输入, 因而在编译过程中内在地引入了一定程度的非确定性 [55]。然而, 执行技能的 LLM 与 CPU 等传统硬件 [23] 有根本区别, 它对输入变化具有内在容忍度, 使技能仍能正确执行。此外, SkVM 严格的编译优化阶段和回滚机制保证了编译后的技能在大多数下游任务上能够持续带来稳定的性能提升。

能力覆盖范围。 当前目录包含分属四类的 26 项原语能力, 足以覆盖我们分析的 15,063 个技能。随着技能库增长, 我们可以采用 §4.1 所述的迭代方法扩展原语能力集合。

编译成本。 AOT 编译会调用 LLM, 因而产生 token 成本。不过, 由于技能会在运行时被反复执行, 编译开销可以在多次调用之间摊销 [21]。此外, 编译后的技能可以在用户与应用之间共享, 进一步降低每个技能的编译成本。

8 结论

技能已经成为代理时代的一种新代码形态。然而, 在分析超过 100,000 个技能后, 我们发现技能与底层 LLM 之间存在显著错配。为此, 我们提出 SkVM, 一个专门面向技能的编译与运行时系统。SkVM 运用 AOT 和 JIT 等编译技术来优化技能结构、提取环境依赖, 并识别并行机会。随着模型与代理框架日益多样化, 技能编译提供了一条系统化路径, 使技能从脆弱的提示词转变为可移植组件。

参考文献

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Agent Skills Initiative. Agent skills specification. <https://agentskills.io/specification>, 2025. Open SKILL.md format for agent skill portability; accessed: 2026-02-06.
- [3] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.
- [4] Anthropic. Claude 3.5 haiku. <https://www.anthropic.com/news/3-5-models-and-computer-use>, 2024. Accessed: 2026-04-02.
- [5] Anthropic. Agent skills. <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2025. Accessed: 2026-02-02.
- [6] Anthropic. Anthropic skills repository. <https://github.com/anthropics/skills>, 2025. Accessed: 2026-03-22.
- [7] Anthropic. Equipping agents for the real world with agent skills. <https://claude.com/blog/equipping-agents-for-the-real-world-with-agent-skills>, 2025. Accessed: 2026-02-06.
- [8] Anthropic. Extend claude with skills. <https://co.de.claude.com/docs/en/skills>, 2025. Claude Code skill mechanism; accessed: 2026-02-06.
- [9] Anthropic. Skill creator. <https://github.com/anthropics/skills/tree/main/skills/skill-creator>, 2025. Accessed: 2026-03-23.
- [10] Anthropic. Claude 4.6 opus. <https://www.anthropic.com/claude/opus>, 2026. Accessed: 2026-04-02.
- [11] Anthropic. Claude code. <https://github.com/anthropics/claude-code>, 2026. Accessed: 2026-04-01.
- [12] John Aycock. A brief history of just-in-time. *ACM computing surveys (CSUR)*, 35(2):97–113, 2003.
- [13] ClawHub. Clawhub: Agent skills marketplace. <https://clawhub.ai>, 2025. Accessed: 2026-03-22.
- [14] Coze. Plugin development guide. https://www.coze.com/docs/guides/plugin?_lang=en, 2025. Coze bot plugins and skills; accessed: 2026-02-06.
- [15] Timothy Cramer, Richard Friedman, Terrence Miller, David Seberger, Robert Wilson, and Mario Wolczko. Compiling java just in time. *Ieee micro*, 17(3):36–43, 1997.
- [16] Cursor. Agent skills. <https://cursor.com/docs/context/skills>, 2025. Cursor IDE skill mechanism; accessed: 2026-02-06.

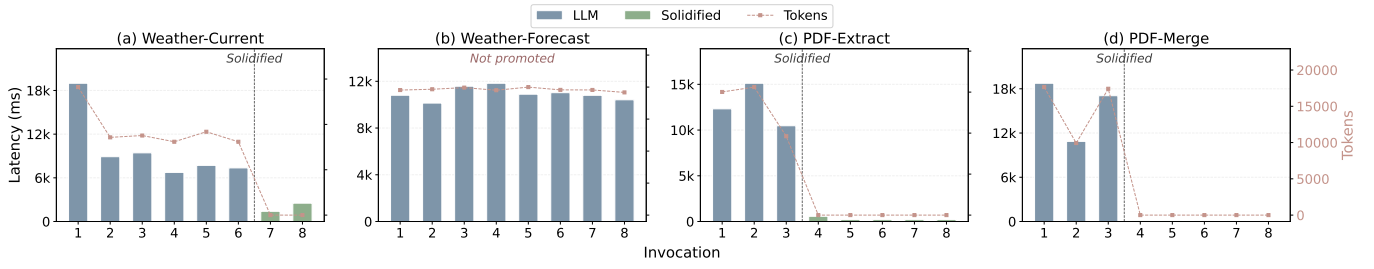


图 15: 四个案例中的代码固化。蓝色柱表示 LLM 推理延迟, 绿色柱表示固化后的执行延迟; 虚线标出晋升点。Weather-forecast 从未晋升, 因为运行时模型的代码模式偏离 AOT 预测, 这验证了晋升门控作为安全机制的作用。

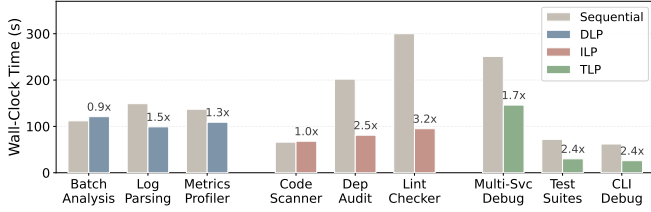


图 16: 八个任务上不同并行策略的性能。柱状图展示顺序执行、DLP、ILP 和 TLP。

- [17] DeepMind. Our latest gemini 3 model that helps you bring any idea to life - faster. <https://deepmind.google/models/gemini/flash/>, 2026. Accessed: 2026-04-02.
- [18] FreedomIntelligence. Openclaw medical skills. <https://github.com/FreedomIntelligence/OpenClaus-Medical-Skills>, 2025. Accessed: 2026-03-23.
- [19] Google. Agent skills. <https://antigravity.google/docs/skills>, 2025. Google Antigravity coding agent skills; accessed: 2026-02-06.
- [20] Google. Agent2agent (a2a) protocol. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interopability/>, 2025. Accessed: 2026-02-06.
- [21] Jim Gray. The transaction concept: Virtues and limitations. *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, pages 144–154, 1981.
- [22] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. Swe-skills-bench: Do agent skills actually help in real-world software engineering? *arXiv preprint arXiv:2603.15401*, 2026.
- [23] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.
- [24] Kazuaki Ishizaki, Motohiro Kawahito, Toshiaki Yasue, Hideaki Komatsu, and Toshio Nakatani. A study of devirtualization techniques for a java just-in-time compiler. In *Proceedings of the 15th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, pages 294–310, 2000.
- [25] Jan Martin Jansen, Pieter Koopman, and Rinus Plasmeijer. From interpretation to compilation. In *Central European Functional Programming School*, pages 286–301. Springer, 2007.
- [26] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [27] Sajith Kalathingal, Caroline Collange, Bharath N Swamy, and André Seznec. Dynamic inter-thread vectorization architecture: extracting dlp from tlp. In *2016 28th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pages 18–25. IEEE, 2016.
- [28] Daniel Kästner and Sebastian Winkel. Ilp-based instruction scheduling for ia-64. *ACM SIGPLAN Notices*, 36(8):145–154, 2001.
- [29] LangChain. Langgraph: Build resilient language agents as graphs. <https://github.com/langchain-ai/langgraph>, 2024. Accessed: 2026-03-22.
- [30] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [31] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [32] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023. arXiv:2107.13586, 2021.
- [33] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

- [34] Mistral AI. Devstral small 2507. <https://huggingface.co/mistralai/Devstral-Small-2507>, 2026. Accessed: 2026-04-02.
- [35] OpenAI. Function calling and other api updates. <https://openai.com/index/function-calling-and-other-api-updates/>, 2023. Accessed: 2026-02-06.
- [36] OpenAI. Openai codex. <https://developers.openai.com/codex>, 2025. Accessed: 2026-02-06.
- [37] OpenClaw. Openclaw: Personal ai assistant. <https://github.com/openclaw/openclaw>, 2025. Accessed: 2026-03-22.
- [38] OpenCode. Opencode: Open-source ai coding agent. <https://github.com/opencode-ai/opencode>, 2025. Accessed: 2026-03-22.
- [39] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022.
- [40] PinchBench. Pinchbench: Benchmarking llm models as coding agents. <https://github.com/pinchbench/skill>, 2026. Accessed: 2026-02-06.
- [41] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations (ICLR)*, 2024.
- [42] Qwen. Qwen-3-30b. <https://huggingface.co/Qwen/Qwen3-30B-A3B>, 2026. Accessed: 2026-04-02.
- [43] Qwen. Qwen-3.5-122b. <https://huggingface.co/Qwen/Qwen3.5-122B-A10B>, 2026. Accessed: 2026-04-02.
- [44] Qwen. Qwen-3.5-397b. <https://huggingface.co/Qwen/Qwen3.5-397B-A17B>, 2026. Accessed: 2026-04-02.
- [45] Narayan Ranganathan and Manoj Franklin. An empirical study of decentralized ilp execution models. *ACM SIGPLAN Notices*, 33(11):272–281, 1998.
- [46] Joshua Redstone, Susan Eggers, and Henry Levy. Mini-threads: Increasing tlp on small-scale smt processors. In *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings.*, pages 19–30. IEEE, 2003.
- [47] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [48] Manuel Serrano. Javascript aot compilation. *ACM SIGPLAN Notices*, 53(8):50–63, 2018.
- [49] Manuel Serrano. Of javascript aot compilation performance. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–30, 2021.
- [50] Skills.sh. Skills.sh: Agent skills registry. <https://skills.sh>, 2025. Accessed: 2026-03-22.
- [51] Bjarne Stroustrup. *The C++ programming language*. Pearson Education, 2013.
- [52] Toshio Sukanuma, Takeshi Ogasawara, Mikio Takeuchi, Toshiaki Yasue, Motohiro Kawahito, Kazuaki Ishizaki, Hideaki Komatsu, and Toshio Nakatani. Overview of the ibm java just-in-time compiler. *IBM systems Journal*, 39(1):175–193, 2000.
- [53] Bill Venners. *Inside the Java Virtual Machine*. McGraw-Hill, 1998.
- [54] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [55] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, et al. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations (ICLR)*, 2023.
- [56] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *Conference on Language Modeling (COLM)*, 2024.
- [57] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [58] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.
- [59] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.